is the approach many apps will need, especially line-of-business apps. Luckily, there's nothing conceptually difficult to it; it requires a simple UI that lets you view users and add a claim through the same mechanism I'll show here.

TIP Another common approach is to customize the `IdentityUser` entity, by adding a `Name` property, for example. This approach is sometimes easier to work with if you want to give users the ability to edit that property. Microsoft's "Add, download, and delete custom user data to Identity in an ASP.NET Core project" documentation describes the steps required to achieve that: http://mng.bz/aoe7.

Let's say you want to add a new `Claim` to a user, called `FullName`. A typical approach would be as follows:

1. Scaffold the Register.cshtml Razor Page, as you did in section 23.5.
2. Add a Name field to the `InputModel` in the Register.cshtml.cs `PageModel`.
3. Add a Name input field to the Register.cshtml Razor view template.
4. Create the new `ApplicationUser` entity as before in the `OnPost()` page handler by calling `CreateAsync` on `UserManager<ApplicationUser>`.
5. Add a new `Claim` to the user by calling `UserManager.AddClaimAsync()`.
6. Continue the method as before, sending a confirmation email or signing the user in if email confirmation is not required.

Steps 1-3 are fairly self-explanatory and require only updating the existing templates with the new field. Steps 4-6 take place in Register.cshtml.cs in the `OnPostAsync()` page handler, which is summarized in the following listing. In practice, the page handler has more error checking, boilerplate, extra features, and abstraction. I've simplified the code in listing 23.7 to focus on the additional